

04 详细设计与模块化复习讲义

目录

4. 详细设计、模块化与信息隐藏	2
4.1 详细设计的范围	2
4.2 责任与协作	3
4.3 详细设计过程	3
4.4 GRASP 原则	4
Low Coupling 低耦合	4
High Cohesion 高内聚	4
Information Expert 信息专家	4
Creator 创建者	5
Controller 控制器	5
4.5 控制风格	5
集中式控制	5
委托式控制	6
分散式控制	6
4.6 Parnas 信息隐藏	6
4.7 模块化	6
4.8 耦合	7
访问耦合	7
Law of Demeter	8
4.9 接口隔离与面向接口编程	8

目录	2
4.10 继承耦合	9
4.11 LSP	10
4.12 组合优于继承	10
4.13 封装	11
4.14 设计原则 P1-P14	11
4.15 集成测试与 Mock	12
对应考试模板	13
7.8 设计原则题模板	13
快速背诵清单	13
详细设计	13

范围：04-01-详细设计.pdf、04-02-面向对象的模块化与信息隐藏.pdf；结合 Exam/Review 中 GRASP、信息隐藏、耦合、内聚、LSP、DIP 和 OCP 内容。
结构：先整理 PPT 主线，再放对应考试模板和快速背诵清单。

4. 详细设计、模块化与信息隐藏

4.1 详细设计的范围

详细设计包含：

- 中层设计：过程、调用关系、类协作。
- 低层设计：数据结构、算法、类型、语句和控制结构。

输入：

- 用例
- 概念模型
- 顺序图
- 状态图
- 非功能需求
- 体系结构模块规格
- 导入和导出接口
- 组件接口

输出：

- 静态结构：类、接口、关系。
- 动态行为：对象协作和消息。
- 状态行为：对象生命周期。
- 实现注解：约束、异常、事务。
- 设计理由：关键选择的依据。

4.2 责任与协作

面向对象详细设计的核心是：

- 责任
- 协作

责任：

- 操作责任：对象要执行的任务。
- 数据责任：对象要维护的信息。

协作：

- 对象之间通过消息调用完成更大的任务。
- 一个对象不直接承担全部工作，而是把子任务委托给合适对象。

委托的意义：

- 降低耦合。
- 分离关注点。
- 让职责落在拥有信息的对象上。

4.3 详细设计过程

可按以下顺序答题：

1. 根据用例和系统事件识别控制入口。
2. 根据概念类图分配数据责任。
3. 根据顺序图分配操作责任。
4. 明确类、接口、属性、方法和关系。
5. 用 GRASP 原则检查职责分配。
6. 用模块化、信息隐藏和设计模式改进结构。
7. 设计单元测试和集成测试。

4.4 GRASP 原则

Low Coupling 低耦合

类之间依赖越少，修改传播越小。

做法：

- 依赖接口。
- 避免全局变量。
- 避免对象链式访问。
- 避免跨层调用。

High Cohesion 高内聚

一个类的职责要集中，成员之间围绕同一目标。

坏例子：

- 一个 Manager 同时处理用户、订单、支付、日志、报表。

好例子：

- RechargeService 处理充值业务。
- PaymentGateway 处理支付平台交互。
- RechargeRecordRepository 处理充值记录持久化。

Information Expert 信息专家

把责任分配给拥有完成该责任所需信息的类。

示例：

- 判断校园卡余额是否足够消费，责任应放在 CampusCard 或管理校园卡余额的服务中，而不是界面对象中。

Creator 创建者

B 创建 A 的条件：

- B 聚合或包含 A。
- B 记录 A。
- B 紧密使用 A。
- B 拥有初始化 A 的数据。

示例：

- `RechargeService` 拥有充值金额、校园卡、支付结果等信息，负责创建 `RechargeRecord`。

Controller 控制器

控制器负责接收系统事件并协调业务对象。

控制器不是所有业务逻辑的堆放点。Controller 接收请求并调用 Service；Service 承担业务流程；领域对象承担自身规则。

4.5 控制风格

集中式控制

少数控制对象掌握大量逻辑。

优点：

- 决策位置集中。
- 流程容易追踪。

缺点：

- 控制器臃肿。
- 其他对象退化为数据容器。

委托式控制

控制器保留主流程，把子任务委托给专业对象。

这是实际项目中较稳妥的风格。

分散式控制

控制逻辑分散在大量小对象中。

问题：

- 控制流难追踪。
- 调试和维护成本高。

4.6 Parnas 信息隐藏

Parnas 的核心思想：

模块隐藏一个重要的设计决策。

模块不只是子程序。模块边界要围绕“未来容易变化的设计决策”设置。

例子：

- 支付平台接口会变化，应隐藏在 PaymentGateway 接口背后。
- 数据库表结构会变化，应隐藏在 Repository 背后。
- 余额更新事务策略会变化，应隐藏在 Service 内部。

4.7 模块化

模块是可以被独立设计、实现和测试的自包含单元。

面向对象中的粒度：

- 方法
- 类
- 包

模块化目标：

- 降低耦合。
- 提高内聚。
- 隐藏变化。
- 支持独立测试。
- 支持并行开发。

4.8 耦合

耦合表示模块之间依赖强度。耦合越低，修改传播越小。

面向对象中两类主要耦合：

- 访问耦合
- 继承耦合

访问耦合

从高到低：

1. 隐式访问耦合：全局变量、单例共享状态。
2. 实现访问耦合：通过反射等方式访问内部实现。
3. 成员变量访问耦合：直接访问 public 字段。
4. 参数变量访问耦合：通过参数传递具体类型。
5. 无访问耦合。

高风险层级：

- 隐式访问耦合
- 实现访问耦合
- 成员变量访问耦合

可接受结构：

- 通过参数和接口协作。
- 不访问对方内部结构。

Law of Demeter

“只和直接朋友交谈”。

一个方法只调用：

- 自身对象的方法。
- 参数对象的方法。
- 本方法创建对象的方法。
- 本对象直接字段的方法。

避免：

```
order.getCustomer().getAddress().getCity()
```

改进方向：

```
order.getCustomerCity()
```

或把责任放到合适对象中。

4.9 接口隔离与面向接口编程

面向接口编程：

- 客户端依赖抽象接口。
- 实现类可以替换。
- 测试时可注入 Mock。

接口隔离原则：

客户端不应被迫依赖它不使用的方法。

坏接口：

```
public interface CampusCardService {  
    void recharge();  
    void reportLost();  
    void generateFinanceReport();  
    void repairDevice();  
}
```

改进：

```
public interface RechargeService {  
    void recharge();  
}
```

```
public interface LossReportService {  
    void reportLost();  
}
```

4.10 继承耦合

继承是白盒复用，子类依赖父类内部设计。

继承耦合类型从高到低：

- Modification：子类修改父类语义，风险最高。
- Refinement：子类细化父类行为。
- Extension：子类增加新能力。
- Nil：无继承耦合。

优先选择：

- 组合
- 委托
- 接口

4.11 LSP

LSP 要求：

子类型对象替换父类型对象后，程序正确性保持不变。

契约要求：

- 子类前置条件不能更强。
- 子类后置条件不能更弱。
- 子类必须维护父类不变式。

例子：

- 如果父类 `Account.withdraw(amount)` 允许在余额足够时取款，子类不能额外要求“amount 必须大于 100”。

4.12 组合优于继承

组合通过对象协作复用行为。

优点：

- 运行时可替换。
- 不暴露父类内部实现。
- 更易遵守 LSP。
- 更容易测试。

示例：

- `RechargeService` 组合 `PaymentGateway`，而不是继承某个支付实现类。

4.13 封装

封装层次：

- A：封装数据类型或抽象数据类型。
- B：封装内部结构。
- C：封装其他对象引用。
- D：封装运行时类型信息。
- E：封装潜在变化。

Iterator 模式就是封装集合内部结构，让客户端不用知道底层是 List、Set 还是 Map。

4.14 设计原则 P1-P14

编号	原则	要点
P1	全局变量有害	全局状态导致隐式依赖
P2	显式表达	依赖、状态、约束要清楚
P3	DRY	不重复知识和逻辑
P4	面向接口编程	依赖抽象而非实现
P5	Law of Demeter	只和直接朋友交谈
P6	ISP	接口保持小而专用
P7	LSP	子类可替换父类
P8	组合优于继承	降低白盒复用风险
P9	封装变化	把变化点藏到稳定接口背后
P10	最小权限	只暴露必要访问权限
P11	OCP	对扩展开放，对修改关闭
P12	DIP	高层和低层都依赖抽象
P13	SRP	一个模块只有一个变化理由
P14	SOFA	短、单一、少参数、抽象层一致

SOFA：

- Short
- One thing
- Few arguments
- Abstraction level consistency

4.15 集成测试与 Mock

集成测试检查类之间的协作。

输入：

- 对象组合
- 触发消息

输出：

- 返回值
- 最终对象状态
- 外部协作是否被调用

Mock 对象用于替代外部依赖：

- 数据库
- 支付平台
- 第三方 API
- 消息队列

充值服务测试示例：

- 注入 Mock PaymentGateway 返回支付成功。
 - 注入 Mock CampusCardRepository 返回状态正常的校园卡。
 - 调用 recharge。
 - 断言余额更新、记录保存、返回结果正确。
-
-

对应考试模板

7.8 设计原则题模板

题目给出代码后，按以下结构答：

1. 指出问题。
2. 对应原则。
3. 说明风险。
4. 给出改进。

示例：

问题：Controller 直接访问 Repository，并在 Controller 中完成充值业务规则。

原则：违反分层职责、低耦合和 SRP。

风险：界面入口与数据库访问耦合，业务规则无法复用，单元测试困难。

改进：Controller 只依赖 RechargeService；

RechargeServiceImpl 注入 Repository 并封装充值流程。

快速背诵清单

详细设计

- 核心：责任与协作。
- GRASP：低耦合、高内聚、信息专家、创建者、控制器。
- 信息隐藏：模块隐藏重要设计决策。
- SOFA：短、单一、少参数、抽象层一致。